XG Boost result evaluation

Library used:

```
# 1. Imports
import dask.dataframe as dd
from dask.distributed import Client
import xgboost as xgb
from dask_ml.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_auc_score
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Starting dask client:

why to initiate dask client

Enables parallel and distributed computation

- Without a client, Dask runs in a simple single-threaded mode, which limits performance.
- With a client, Dask can:
    - Run tasks in parallel on multiple CPU cores.
    - Scale out to multiple machines if available.
    - Efficiently schedule and balance work dynamically.

```
# 2. Start Dask client
client = Client()
print(client)
```

```
/usr/local/lib/python3.12/dist-packages/distributed/node.py:188: UserWarning: Port 8787 is already in use.
Perhaps you already have a cluster running?
Hosting the HTTP server on port 33857 instead
  warnings.warn(
INFO:distributed.scheduler:State start
INFO:distributed.scheduler:  Scheduler at:     tcp://127.0.0.1:43727
INFO:distributed.scheduler:  dashboard at:  http://127.0.0.1:33857/status
INFO:distributed.scheduler:Registering Worker plugin shuffle
INFO:distributed.nanny:        Start Nanny at: 'tcp://127.0.0.1:35827'
INFO:distributed.nanny:        Start Nanny at: 'tcp://127.0.0.1:35607'
INFO:distributed.scheduler:Register worker addr: tcp://127.0.0.1:40989 name: 1
INFO:distributed.scheduler:Starting worker compute stream, tcp://127.0.0.1:40989
INFO:distributed.core:Starting established connection to tcp://127.0.0.1:53588
INFO:distributed.scheduler:Register worker addr: tcp://127.0.0.1:36627 name: 0
INFO:distributed.scheduler:Starting worker compute stream, tcp://127.0.0.1:36627
INFO:distributed.core:Starting established connection to tcp://127.0.0.1:53602
INFO:distributed.scheduler:Receive client connection: Client-083022a4-16f9-11f1-9005-0242ac1c000c
INFO:distributed.core:Starting established connection to tcp://127.0.0.1:53612
<Client: 'tcp://127.0.0.1:43727' processes=2 threads=2, memory=12.67 GiB>
```

Loading dataset and initial transformation: dataset is loaded and datatype of few of the columns in specified

```python
# 3. Load dataset
# Columns: order_id, user_id, order_number, product_name, reordered
df = dd.read_csv(
    "full_file.csv",
    dtype={
        'add_to_cart_order': 'float64',
        'order_dow': 'float64',
        'order_hour_of_day': 'float64',
        'order_number': 'float64',
        'product_id': 'float64',
        'reordered': 'float64'
    }
)
```

Total number of records:

```python
print(df.shape[0].compute())
```

```
11160636
```

Structure of dataset

```python
df.head()
```

	order_id	user_id	eval_set	order_number	order_dow	order_hour_of_day	days_since_prior_order	product_id	add_to_cart_order	reordered	product_name	aisle	department
0	473747	1	prior	3.0	3.0	12.0	21.0	10258.0	3.0	1.0	Pistachios	nuts seeds dried fruit	snacks
1	473747	1	prior	3.0	3.0	12.0	21.0	25133.0	4.0	0.0	Organic String Cheese	packaged cheese	dairy eggs
2	473747	1	prior	3.0	3.0	12.0	21.0	30450.0	5.0	0.0	Creamy Almond Butter	spreads	pantry
3	431534	1	prior	5.0	4.0	15.0	28.0	196.0	1.0	1.0	Soda	soft drinks	beverages
4	431534	1	prior	5.0	4.0	15.0	28.0	12427.0	2.0	1.0	Original Beef Jerky	popcorn jerky	snacks

Selecting the fields to work with and initial transformation of data

```python
df_small = df[['order_id', 'user_id', 'product_name']]
```

```python
df_small.head()
```

	order_id	user_id	product_name
0	473747	1	Pistachios
1	473747	1	Organic String Cheese
2	473747	1	Creamy Almond Butter
3	431534	1	Soda
4	431534	1	Original Beef Jerky

```python
df_small['product_name'] = df_small['product_name'].str.lower()
```

Categorizing the columns, setting datatype and viewing final output

```python
df = (
    df_small
    .categorize(columns=['user_id', 'product_name'])
    .persist()
)
```

```python
df = df.astype({
    "user_id": "int64",
    "order_id": "int64",
    "product_name": "string"
})
```

```python
df.head()
```

	order_id	user_id	product_name
0	473747	1	pistachios
1	473747	1	organic string cheese
2	473747	1	creamy almond butter
3	431534	1	soda
4	431534	1	original beef jerky

Grouping based on user and order and displaying the product present in that order as a python list:

```python
# Step 1: Compute and aggregate products per order for each user
orders_products = df.groupby(['user_id', 'order_id'])['product_name'].apply
(list).compute()
```

```
/tmp/ipykernel_53253/3961388644.py:2: UserWarning: `meta` is not specified, inferred from partial data.
Please provide `meta` if the result is unexpected.
  Before: .apply(func)
  After:  .apply(func, meta={'x': 'f8', 'y': 'f8'}) for dataframe result
  or:     .apply(func, meta=('x', 'f8'))            for series result

  orders_products = df.groupby(['user_id', 'order_id'])['product_name'].apply(list).compute()
WARNING:distributed.shuffle._scheduler_plugin:Shuffle 07233520dbbe813e4a775b65305d3ba4 initialized by task ('shuffle-transfer-07233520dbbe813e4a775b65305d3ba4', 4) executed on worker tcp://127.0.0.1:40989
WARNING:distributed.shuffle._scheduler_plugin:Shuffle 07233520dbbe813e4a775b65305d3ba4 deactivated due to stimulus 'task-finished-1772540023.3867805'
```

```python
orders_products = orders_products.reset_index()
```

```python
orders_products.head()
```

	user_id	order_id	product_name
0	32	72626	[organic snipped green beans, single serve cri...
1	32	134879	[organic smoked turkey breast, organic thyme, ...
2	87	88090	[milk, organic, vitamin d]
3	87	122960	[organic large brown grade aa cage free eggs, ...
4	87	161814	[bread, country buttermilk, original low fat o...

Sorting based on user and order id and comparing what is being ordered in penultimate and last order

```python
# Step 2: Sort orders by user and order_id to find next order info
orders_products = orders_products.sort_values(['user_id', 'order_id'])

# Add next order_id and next order products per user
orders_products['next_order_id'] = orders_products.groupby('user_id')
['order_id'].shift(-1)
orders_products['next_products'] = orders_products.groupby('user_id')
['product_name'].shift(-1)
```

```python
orders_products.head()
```

	user_id	order_id	product_name	next_order_id	next_products
348337	1	431534	[soda, original beef jerky, pistachios, organi...	473747.0	[pistachios, organic string cheese, creamy alm...
348338	1	473747	[pistachios, organic string cheese, creamy alm...	550135.0	[original beef jerky, organic string cheese, c...
348339	1	550135	[original beef jerky, organic string cheese, c...	1187899.0	[soda, xl pick-a-size paper towel rolls, pista...
348340	1	1187899	[soda, xl pick-a-size paper towel rolls, pista...	NaN	NaN
140045	2	738281	[chipotle beef & pork realstick, hommus classi...	788338.0	[organic hass avocado, blueberry yoghurt, rasp...

Exploding the above data frame and resetting the index

```python
# Step 3: Explode products in current order to get one row per product
orders_products_expanded = orders_products.explode('product_name').reset_index
(drop=True)
```

```python
orders_products_expanded.head()
```

	user_id	order_id	product_name	next_order_id	next_products
0	1	431534	soda	473747.0	[pistachios, organic string cheese, creamy alm...
1	1	431534	original beef jerky	473747.0	[pistachios, organic string cheese, creamy alm...
2	1	431534	pistachios	473747.0	[pistachios, organic string cheese, creamy alm...
3	1	431534	organic fuji apples	473747.0	[pistachios, organic string cheese, creamy alm...
4	1	431534	honeycrisp apples	473747.0	[pistachios, organic string cheese, creamy alm...

Introducing a new column “label” based on a function that will evaluate 1 if the product in penultimate order also exists in last order else 0

```python
# Step 4: Define label: 1 if product appears in next order, else 0
def label(row):
    # If product_name is missing, label 0
    if pd.isna(row['product_name']):
        return 0

    next_prods = row['next_products']

    # Handle missing / non-iterable next_products
    if not isinstance(next_prods, (list, tuple, set, pd.Series, np.ndarray)):
        return 0

    # Convert to list safely
    next_list = list(next_prods)

    # Safe membership check
    return 1 if row['product_name'] in next_list else 0

orders_products_expanded['label'] = orders_products_expanded.apply(label,
axis=1)
```

```
INFO:distributed.core:Event loop was unresponsive in Nanny for 4.18s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Scheduler for 4.20s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Nanny for 4.21s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Scheduler for 3.49s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Nanny for 3.52s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Nanny for 3.53s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Scheduler for 3.07s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Nanny for 3.08s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Nanny for 3.51s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Nanny for 11.14s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Scheduler for 9.24s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Nanny for 11.16s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Scheduler for 8.16s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Nanny for 7.72s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
INFO:distributed.core:Event loop was unresponsive in Nanny for 8.16s.  This is often caused by long-running GIL-holding functions or moving large chunks of data. This can cause timeouts and instability.
```

```
orders_products_expanded.head()
```

	user_id	order_id	product_name	next_order_id	next_products	label
0	1	431534	soda	473747.0	[pistachios, organic string cheese, creamy alm...	1
1	1	431534	original beef jerky	473747.0	[pistachios, organic string cheese, creamy alm...	1
2	1	431534	pistachios	473747.0	[pistachios, organic string cheese, creamy alm...	1
3	1	431534	organic fuji apples	473747.0	[pistachios, organic string cheese, creamy alm...	0
4	1	431534	honeycrisp apples	473747.0	[pistachios, organic string cheese, creamy alm...	0

Introducing categorical feature:

Why it is done

This step is all about preparing your data for machine learning models by converting categorical text data into numeric codes.

- Takes the product_name and user_id columns (which are usually strings or objects).
- Converts them to pandas categorical type.
- Then extracts the integer codes representing each unique category.
- Stores those integer codes in new columns product_cat and user_cat.

ML models need numbers, not strings:

- Most machine learning algorithms (like XGBoost) cannot use raw text or string data directly.
- They need numeric input features.

Efficient representation of categories:

- Instead of arbitrary strings like 'user_123' or 'apple', you get numeric IDs like 0, 1, 2, ...
- This makes data compact and fast to process.

Avoids exploding feature space

- Compared to one-hot encoding (which creates a binary column per category), using .cat.codes is:
    - More memory-efficient
    - Keeps a fixed-size integer feature per category
- Particularly good when you have lots of categories (many users/products).

Keeps unique mappings

- Each unique user or product gets a unique integer ID.
- This is crucial for models to learn distinct patterns for different users/products.

Important note

- These integer codes do not imply any order or ranking.
- They are just labels, so 0 does not mean "less than" 1.
- Many tree-based models handle this just fine.

Summary

You convert user and product IDs from strings into numeric codes so your machine learning model can process them efficiently and correctly.

```
# Step 5: Encode categorical features
orders_products_expanded['product_cat'] = orders_products_expanded
['product_name'].astype('category').cat.codes
orders_products_expanded['user_cat'] = orders_products_expanded['user_id'].
astype('category').cat.codes
```

```
orders_products_expanded.head()
```

	user_id	order_id	product_name	next_order_id	next_products	label	product_cat	user_cat
0	1	431534	soda	473747.0	[pistachios, organic string cheese, creamy alm...	1	40577	0
1	1	431534	original beef jerky	473747.0	[pistachios, organic string cheese, creamy alm...	1	32137	0
2	1	431534	pistachios	473747.0	[pistachios, organic string cheese, creamy alm...	1	34359	0
3	1	431534	organic fuji apples	473747.0	[pistachios, organic string cheese, creamy alm...	0	29407	0
4	1	431534	honeycrisp apples	473747.0	[pistachios, organic string cheese, creamy alm...	0	20000	0

Splitting the data into test and train

Data is split in such a way that a user if appearing in test will not appear in train, this is done to avoid data leakage between test and train

```
import numpy as np
from sklearn.model_selection import train_test_split

# Step 6: SPLIT USERS TO AVOID LEAKAGE

# 1 Ensure user_id is clean: drop missing values and convert to string
orders_products_expanded['user_id_str'] = orders_products_expanded['user_id'].
astype(str)

# 2 Get unique users as a clean NumPy array
unique_users = orders_products_expanded['user_id_str'].dropna().unique()
unique_users = np.array(unique_users, dtype=str)  # explicitly make 1D array of
strings

# 3 Split users into train and test
train_users, test_users = train_test_split(
    unique_users,
    test_size=0.2,
    random_state=42,
    shuffle=True
)

# 4 Filter the dataset by user_id to create train and test sets
train_df = orders_products_expanded[orders_products_expanded['user_id_str'].isin
(train_users)].copy()
test_df = orders_products_expanded[orders_products_expanded['user_id_str'].isin
(test_users)].copy()

# Optional: Drop the helper column if you want
train_df = train_df.drop(columns=['user_id_str'])
test_df = test_df.drop(columns=['user_id_str'])

# ✅ Now train_df and test_df have no user leakage
```

Train data frame

```
train_df.head()
```

	user_id	order_id	product_name	next_order_id	next_products	label	product_cat	user_cat
0	1	431534	soda	473747.0	[pistachios, organic string cheese, creamy alm...	1	40577	0
1	1	431534	original beef jerky	473747.0	[pistachios, organic string cheese, creamy alm...	1	32137	0
2	1	431534	pistachios	473747.0	[pistachios, organic string cheese, creamy alm...	1	34359	0
3	1	431534	organic fuji apples	473747.0	[pistachios, organic string cheese, creamy alm...	0	29407	0
4	1	431534	honeycrisp apples	473747.0	[pistachios, organic string cheese, creamy alm...	0	20000	0

Test data frame

```
test_df.head()
```

	user_id	order_id	product_name	next_order_id	next_products	label	product_cat	user_cat
134	5	157374	plain whole milk yogurt	2196797.0	[organic raw agave nectar, sharp cheddar chees...	0	34513	4
135	5	157374	red raspberries	2196797.0	[organic raw agave nectar, sharp cheddar chees...	0	36984	4
136	5	157374	uncured genoa salami	2196797.0	[organic raw agave nectar, sharp cheddar chees...	0	45753	4
137	5	157374	organic grape tomatoes	2196797.0	[organic raw agave nectar, sharp cheddar chees...	1	29563	4
138	5	157374	large organic omega3 brown eggs	2196797.0	[organic raw agave nectar, sharp cheddar chees...	0	21991	4

Preparing test and train data:

```
# Step 7: Prepare train/test data
X_train = train_df[['product_cat', 'user_cat']]
y_train = train_df['label']

X_test = test_df[['product_cat', 'user_cat']]
y_test = test_df['label']
```

Training the model:

Scaling is done to make model more sensitive to reorder as

y_train is your target:

- 1 → product was reordered in next order
- 0 → product was not reordered

In most datasets, most products are not reordered → 0 dominates

This makes your dataset highly imbalanced:

Why it's important

Without scaling:

- The model sees many more 0s than 1s
- It may learn to predict 0 for almost everything, because it "wins" most of the time
- Result: very high accuracy but terrible at detecting reorders (minority class)

Scaling makes the model pay more attention to the minority class (1):

- Each positive sample is "worth" more in the loss function
- The model becomes more sensitive to products that are reordered

label	count
0	90%
1	10%

- `scale_pos_weight` calculates:

$$\text{scale_pos_weight} = \frac{\#\text{ of 0s}}{\#\text{ of 1s}}$$

```
# Step 8: Train XGBoost model
scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()# to makeit more
sensitive to rerderasmost are not reordered
model = xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss',
scale_pos_weight=scale_pos_weight)
model.fit(X_train, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [12:34:56] WARNING: /__w/xgboost/xgboost/src/learner.cc:782:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
```

XGBClassifier

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=None, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=None, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=None, n_jobs=None,
```

Predicting and evaluating the model performances:

Test accuracy

```
#Step 9: Predict and evaluate
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")
```

```
Test Accuracy: 0.6305
```

Confusion Matrix

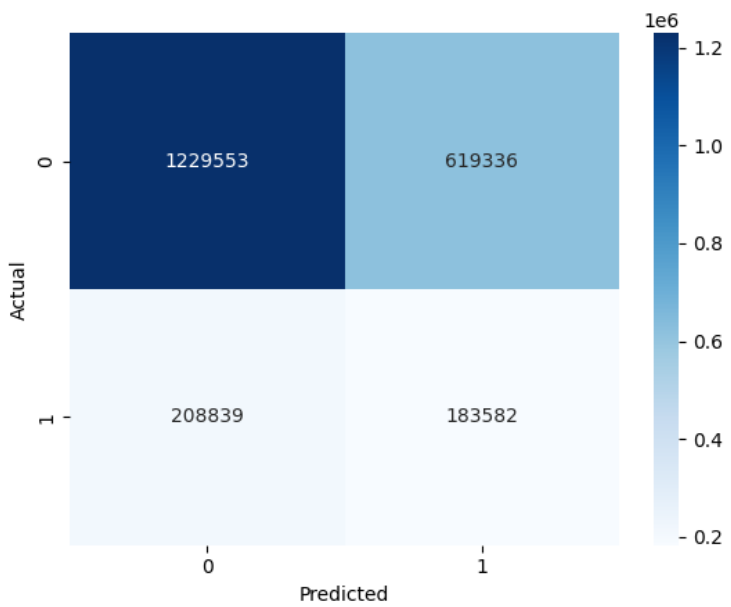
```python
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[1229553  619336]
 [ 208839  183582]]
```

```python
import seaborn as sns

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```




```python
print("ROC AUC:", roc_auc_score(y_test, model.predict_proba(X_test)[:,1]))
```

```
ROC AUC: 0.5928806898888209
```

```python
from sklearn.metrics import precision_score, recall_score, f1_score

precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

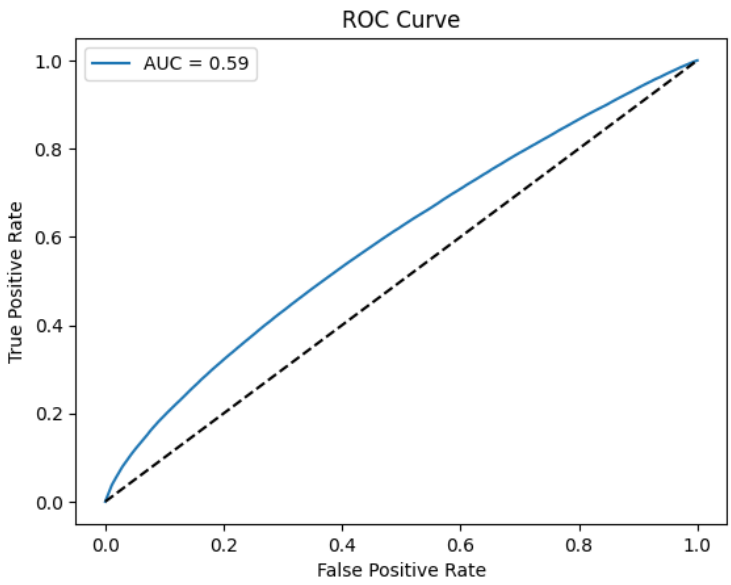
```python
print(precision)
print(recall)
print(f1)
```

```
0.22864352275076658
0.467819000512205
0.30716307256769837
```

```python
from sklearn.metrics import roc_curve, roc_auc_score

probs = model.predict_proba(X_test)[:,1]
fpr, tpr, thresholds = roc_curve(y_test, probs)
roc_auc = roc_auc_score(y_test, probs)

plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.plot([0,1], [0,1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```




```python
#label is repeat rate, fraction of times it was reordered
```

```python
repeat_rates = orders_products_expanded.groupby('product_name')['label'].mean()
repeat_rates = repeat_rates.sort_values(ascending=False)
repeat_rates.head(10)
```

product_name	label
soy crisps lightly salted	0.833333
coffee, ground, medium, special roast	0.800000
organic raspberry black tea	0.750000
orange flavored sports drink	0.750000
100% unsweetened honeysuckle natural iced white tea	0.714286
benchbreak chardonnay	0.702703
chocolate love bar	0.689655
zero peach nutrient enhanced water beverage	0.666667
tropical turmeric jun-kombucha made with honey organic	0.666667
thick sliced bologna	0.666667

dtype: float64

Based on confusion matrix

	Predicted 0	Predicted 1
Actual 0	1,229,553	619,336
Actual 1	208,839	183,582

- True Negatives (TN) = 1,229,553
  Actual = 0 (product not reordered), predicted = 0 (correctly predicted no reorder)
  model correctly identified over 1.2 million non-reorders.
- False Positives (FP) = 619,336
  Actual = 0, predicted = 1
  model predicted a reorder when there wasn't one (over-predicted reorders).
- False Negatives (FN) = 208,839
  Actual = 1 (product reordered), predicted = 0
  model missed about 208k products that were reordered (under-predicted reorders).
- True Positives (TP) = 183,582
  Actual = 1, predicted = 1
  model correctly predicted about 183k products that were reordered.

Overall model performance

- The model is very good at identifying non-reorders (TN much larger than FP).
- But it has a fair number of false positives (~619k), which means it sometimes predicts reorders when there aren't any.
- Also, it misses many true reorders (FN > TP), indicating recall is limited.

Imbalanced dataset reflected

- Since the number of actual 0s (non-reorders) is much larger than 1s (reorders), TN and FP counts are naturally higher.
- The model struggles to perfectly separate positive reorder cases.

Precision:

About 22.8% of predicted reorders were actually correct.

Recall (sensitivity to reorders):

The model detected about 46.8% of actual reorders.

Metric	Value	Meaning
True Negative	1.2M+	Correctly predicted no reorder
False Positive	619k	Predicted reorder but wasn't (over-prediction)
False Negative	208k	Missed actual reorders
True Positive	183k	Correctly predicted reorder
Precision	~23%	Low — many false reorder predictions
Recall	~47%	Moderate — detects less than half of reorders

What does ROC and AUC curve indicate

What the ROC Curve shows

- ROC (Receiver Operating Characteristic) curve plots:
    - True Positive Rate (TPR) aka Recall (y-axis)
    - False Positive Rate (FPR) (x-axis)
- It shows the trade-off between correctly identifying positive cases (reorders) and falsely labelling negatives as positives, as you vary the classification threshold.
- The dashed diagonal line represents a random classifier (AUC = 0.5).

Your plot analysis

- Your curve lies above the diagonal, so your model is better than random guessing.
- AUC = 0.59 means:
    - If you randomly pick one reordered product and one non-reordered product, your model will correctly rank the reordered product higher about 59% of the time.
    - This is only a slightly better than random classifier.

What this means for your model

- AUC = 0.59 is low — ideally, you'd want closer to 0.8 or above for a useful model.
- The model has limited ability to distinguish reordered products from non-reordered ones.
- This matches what we saw in the confusion matrix:
    - Moderate recall but many false positives
    - Not very precise or confident predictions

We have tried to change the threshold to see the impact

```
# identifying better threshold
```

```
from sklearn.metrics import precision_recall_curve

probs = model.predict_proba(X_test)[:,1]
precision, recall, thresholds = precision_recall_curve(y_test, probs)

# Compute F1 scores for all thresholds
f1_scores = 2 * (precision * recall) / (precision + recall + 1e-10)  # small
epsilon to avoid div by zero

# Find threshold with max F1
best_idx = f1_scores.argmax()
best_threshold = thresholds[best_idx]

print(f"Best threshold by F1 score: {best_threshold:.3f}")
```

```
Best threshold by F1 score: 0.473
```

Better threshold is 0.473 instead of 0.5

Applying custom threshold and checking accuracy

```
probs = model.predict_proba(X_test)[:,1]

# Apply custom threshold
custom_threshold = best_threshold  # from step above, e.g., 0.3 or 0.4
y_pred_custom = (probs >= 0.473).astype(int)
```

```
accuracy = accuracy_score(y_test, y_pred_custom)
print(f"Test Accuracy: {accuracy:.4f}")
```

```
Test Accuracy: 0.5321
```

There is a dip in the value

Confusion matrix

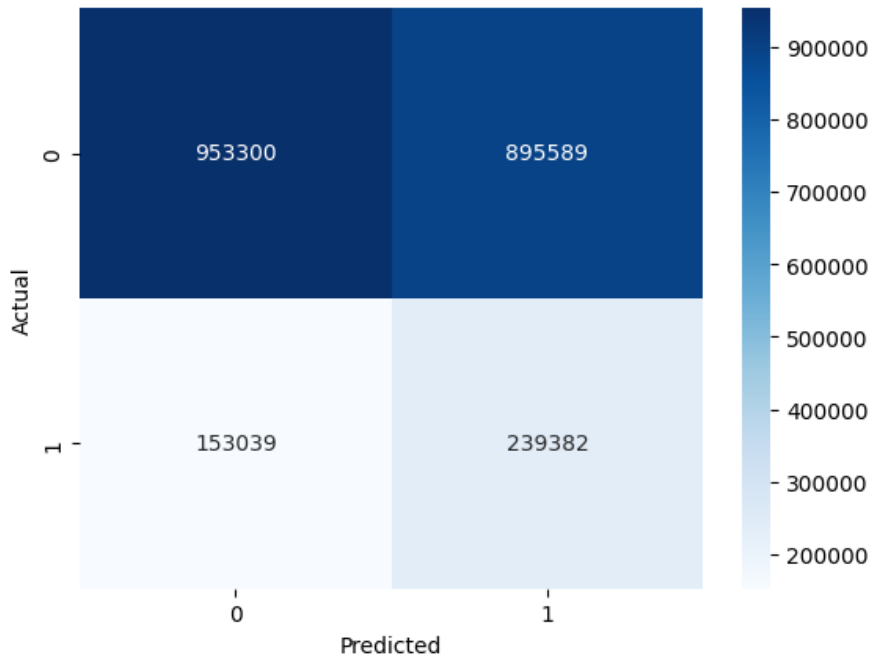
```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred_custom)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[953300 895589]
 [153039 239382]]
```

```python
import seaborn as sns

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



Reevaluation

```python
from sklearn.metrics import precision_score, recall_score, f1_score

precision = precision_score(y_test, y_pred_custom)
recall = recall_score(y_test, y_pred_custom)
f1 = f1_score(y_test, y_pred_custom)
```

```python
print(precision)
print(recall)
print(f1)
```

```
0.2109146401097473
0.6100132255919025
0.31345194946680355
```

Metric	Value	Meaning
Precision	0.21	Only about 21% of predicted reorders were correct (many false positives)
Recall	0.61	The model caught about 61% of all actual reorders (missed 39%)
F1-score	0.31	Harmonic mean of precision and recall — low overall balance

What this means about your model performance:

- Recall improved significantly compared to before (~46.8% from confusion matrix earlier → now 61%).
  This means your model is better at finding actual reorder events, which is often critical in applications where missing a reorder is costly.
- Precision is low (21%), indicating many false positives (predicting reorder when it's not). This is the trade-off for improving recall.
- F1-score increased (0.31) — better than before, showing improved overall balance.
- Accuracy is not shown here but likely decreased or stayed similar, which is expected in imbalanced problems when improving recall.

So, can we say performance improved despite dip in accuracy?

Yes!

- In imbalanced classification, accuracy is often misleading because the majority class dominates.
- Improving recall and F1-score (especially recall) often matters more practically, depending on your business goals.
- If catching more true reorder cases is more valuable than avoiding some false alarms, this is a good trade-off.

How was best threshold evaluated

The best threshold was evaluated using the F1 score calculated at different thresholds from the model's predicted probabilities. Let me explain step by step how it works:

Predicted probabilities

From your trained XGBoost model:

```
probs = model.predict_proba(X_test)[:,1]
```

- This gives the probability that each sample belongs to class 1 (product reordered)
- Values are between 0 and 1

Generate precision and recall at multiple thresholds

```
from sklearn.metrics import precision_recall_curve
precision, recall, thresholds = precision_recall_curve(y_test, probs)
```

- precision[i] and recall[i] correspond to the threshold thresholds[i]
- This lets us see how precision vs recall changes as we move the decision threshold from 0 to 1

Compute F1 score for each threshold

The F1 score is the harmonic mean of precision and recall:

$$F1 = \frac{2 \cdot (precision \cdot recall)}{precision + recall}$$

f1_scores = 2 * (precision * recall) / (precision + recall + 1e-10)

- We add a tiny 1e-10 to avoid division by zero.
- This gives the F1 score at every threshold.

Pick the threshold that maximizes F1

best_idx = f1_scores.argmax()
best_threshold = thresholds[best_idx]

- best_idx is the index of the maximum F1 score
- best_threshold is the probability threshold that gives the best balance between precision and recall

Apply the threshold

y_pred_custom = (probs >= best_threshold).astype(int)

- Instead of using the default 0.5 threshold, you now classify samples as reordered if their predicted probability ≥ best_threshold.
- This improves recall while balancing precision according to F1.